

INTERCEPTING NETWORK FLOWS VIA MIDDLEWARE



MODULE OVERVIEW / LAGOM PANANAW

This module analyzes the execution lifecycle of intercepting network mechanisms within Express.js application frameworks. Learners will examine the syntax boundaries of standard middleware components, establish sequence chaining architectures using active control functions, categorize the distinct behaviors between application-level, router-level, built-in, and third-party utility scripts, and configure robust centralized error-handling pipelines. By building functional interceptor hooks, students establish modular engineering workflows necessary to isolate critical system layers from core functional route logic handlers.



PRETEST / PAUNANG PAGSUBOK

Instructions: Select the best answer for each question. (20 Points)

1. What primary role does a middleware function serve inside an active Express.js runtime structure?
 - A) It compiles JavaScript into binary files.
 - B) It acts as an interceptor for requests and responses before they reach route handlers.
 - C) It maps database schemas to frontend HTML elements.
 - D) It acts as a primary hard drive storage controller.
2. How many parameters does a standard, non-error-handling Express middleware function consume?
 - A) 2 parameters
 - B) 3 parameters
 - C) 4 parameters
 - D) 5 parameters
3. Which method parameter acts as the execution signal to pass control to the subsequent function inside an active middleware stack?
 - A) res.send()
 - B) app.use()
 - C) next()
 - D) req.body
4. What fatal network side effect occurs if a developer forgets to invoke next() at the conclusion of a custom logging middleware function that doesn't send a response?
 - A) The database drops its tables.
 - B) The server reboots into safe mode.
 - C) The client request hangs indefinitely and eventually times out.
 - D) The application changes its active port setting.
5. An error-handling middleware function is uniquely identified by the runtime engine because it is declared with how many parameters?
 - A) Two parameters
 - B) Three parameters
 - C) Four parameters
 - D) One parameter



6. Which variable represents the signature argument layout structure of an official error-handling middleware function?
- A) (req, res, next)
 - B) (err, req, res)
 - C) (err, req, res, next)
 - D) (req, res, next, status)
7. Application-level middleware blocks are explicitly attached to an instance object using which command approach?
- A) router.use()
 - B) app.use()
 - C) npm install
 - D) nodemon index.js
8. Router-level middleware layers distinguish themselves because they are explicitly bound to:
- A) The entire global system machine space
 - B) Third-party packages downloaded from npm
 - C) Specific route segments using router.use() or router.METHOD()
 - D) Pure built-in asset directories
9. Which internal built-in middleware statement is used to serve raw client images, stylesheets, and frontend script files directly?
- A) express.json()
 - B) express.Router()
 - C) express.static()
 - D) express.urlencoded()
10. Third-party packages like morgan or cors are categorized under what architectural tier designation?
- A) Built-in Middleware
 - B) Application-level Middleware
 - C) Custom Middleware
 - D) External/Third-party Middleware
11. What third-party package utility script is specifically chosen to track client requests and format runtime logs without cluttering business logic code?
- A) helmet
 - B) passport.js
 - C) morgan
 - D) express-session
12. Where should a global error-handling middleware function be structurally registered within your application stack file layout?
- A) At the absolute top of the file before declaring express()
 - B) Inside the external script block files of package.json
 - C) At the end of your middleware stack, following all routes and handlers
 - D) Directly enclosed within the initialization arguments of npm init
13. Which command setup statement registers a development engine file watcher that automatically restarts execution when server script lines change?
- A) node index.js
 - B) "start": "nodemon index.js"
 - C) npm init -y
 - D) npm install express



14. Which specific parameter element properties let a developer read incoming network path targets from inside a generic logRequests wrapper function?
- A) req.method and req.url
 - B) res.status and res.body
 - C) err.stack and err.message
 - D) next.function and next.cycle
15. If a security validation middleware function rejects an unauthorized token check request, what HTTP status response code should be transmitted back to the user client?
- A) 200
 - B) 201
 - C) 401
 - D) 500
16. What parameter payload string is explicitly passed to the routing stack inside an error block trigger via next(err)?
- A) The target redirect path rule
 - B) The active variable stack error description object
 - C) The baseline fallback port configuration number
 - D) The raw response payload template text
17. The design quality that allows engineering teams to split up complicated business logic requirements into smaller, distinct, reusable units is defined as:
- A) Stateless validation
 - B) Modularity
 - C) System compression
 - D) Overloading
18. What component acts as the baseline configuration dashboard tracking framework dependency declarations like "express": "^4.19.2"?
- A) app.js
 - B) index.js
 - C) package.json
 - D) node_modules
19. Which built-in utility option allows backend systems to capture inbound form submissions containing encoded text strings?
- A) express.json()
 - B) express.cookieParser()
 - C) express.urlencoded({ extended: true })
 - D) express.csrf()
20. True or False: If a middleware function intentionally calls res.status(401).send(), it must still invoke next() to avoid hanging the network request line.
- A) True
 - B) False



INTRODUCTION / PANIMULA

Have you ever tried accessing a premium streaming platform or checking your bank account balance only to be redirected back to a login screen? Or have you noticed how server teams

instantly record your system's location background info every time you hit a page refresh button?

None of those operations are manually coded into the individual website page features. Instead, they are handled by a series of gatekeepers working behind the scenes. In the Express framework, these gatekeepers are called Middleware Functions. By serving as invisible interceptors along the request-response pathway, middleware blocks keep systems incredibly secure, highly organized, and modular enough to run heavy enterprise web operations smoothly.



LEARNING OBJECTIVES / MGA LAYUNIN SA PAGKATUTO

By the end of this module, students should be able to:

- Differentiate between the distinct functional behaviors of built-in, third-party, and custom middleware layers.
- Implement sequenced middleware chaining scripts to process incoming network traffic requests.
- Create and register global error-handling middleware structures to intercept application exceptions gracefully.



PRESENTATION OF THE LESSON / PAGLALAHAD NG ARALIN

A. Preliminary Activity

Open your system editor workspace. Suppose you are tasked with building a premium content application. If you have ten distinct hidden dashboard paths, you could copy and paste a security verification statement inside all ten routes. Alternatively, you could place a single gatekeeper block right at the main server entry gateway to intercept all inbound traffic. Write down what benefits this entry gate approach introduces regarding code length, maintenance overhead, and bug reduction.

B. Discussion of the Lesson

What is Middleware in Express.js?

Express is a popular Node.js framework celebrated for its ease of use and adaptability in creating web applications and APIs. A key aspect of Express is its middleware, which is crucial for managing requests, responses, and routing.

Plaintext

```
Client -----> [ MIDDLEWARES ] -----> Route Handler
```

Middleware functions act as interceptors for requests and responses before they reach the actual route handlers. These functions are invoked by the routing layer and offer great flexibility by enabling features such as logging, authentication, and error handling.

Code Syntax Rules

The baseline structural execution syntax blueprint is configured as follows:

```
app.get(path, (req, res, next) => {
  // Middleware logic execution space
}, (req, res) => {
  // Terminal route handler logic execution space
});
```

Middleware functions take three arguments: the request object (req), the response object (res), and the next function in the application's request-response cycle. That means it maps out exactly two objects and one function.

Middleware layers execute actions that impact the behavior of the application by modifying properties attached directly to request or response variables. They can also immediately terminate the runtime loop by returning a network response early if security rules fail. If they do not terminate the request-response cycle, they must trigger their third parameter argument, next(), to pass control to the subsequent block in the execution sequence.

Types of Middleware

Express systems split up middleware implementations into five distinct functional categories:

Middleware Category	Scope & Attachment Rules	Typical Core Use Cases
1. Application-level	Bound globally to the whole application via app.use() or app.METHOD() statements.	Runs systematically across every inbound route pathway configured.
2. Router-level	Associated strictly with narrow instances using router.use() or router.METHOD() rules	Restricts execution to specific logical sub-routes defined within that isolated routing workspace
3. Error-handling	Catch-all blocks defined uniquely with four parameters: (err, req, res, next)	Intercepts system crashes and builds friendly failure notifications for users
4. Built-in	Standard libraries packaged out-of-the-box by Express creators	Examples include parsing data payload arrays (express.json()) or hosting static asset files (express.static())
5. Third-party	External functional code modules downloaded from the external npm package registry	Popular examples: Logging utilities (morgan), security frameworks (helmet), and resource sharing rules (cors)

Core Architectural Advantages

Leveraging structured interceptor blocks introduces several structural benefits to production workspaces:

- **Modularity:** Breaks down complex, tangled server architectures into isolated, easily manageable tasks.
- **Separation of Concerns:** Isolates cross-cutting tasks like logging or authorization away from core system business logic.
- **Reusability:** Allows engineering teams to apply identical code rules consistently across multiple distinct pathways.

- Centralized Error Control: Catches application exceptions uniformly without risking abrupt system terminal crashes.

Project Implementation Sequence

To construct a structured application environment tracking middleware behavior, build out the following framework configuration setup:

Step 1: Initialize the Environment

Generate your baseline metadata target files dynamically using the terminal command below:

```
npm init -y
```

Step 2: Install Required Workspace Tools

Mount the main web framework alongside the active live-reloading process watcher utility package:

```
npm install express nodemon
```

Step 3: Configure Automated Scripts

Open your newly initialized package.json file, navigate down to the active scripts node block, and declare the live tracking script statement:

```
"scripts": {  
  "start": "nodemon index.js"  
}
```

Step 4: Write the Application Core Script

Create index.js inside your editor workspace and declare baseline path routing mechanics:

```
const express = require("express");  
const app = express();  
const port = process.env.port || 3000;  
  
// Root testing pathway layout rule  
app.get("/", (req, res) => {  
  res.send(`<div>  
    <h2>Welcome to Alibaba</h2>  
    <h5>Tutorial on Middleware</h5>  
  </div>`);  
});  
  
app.listen(port, () => {  
  console.log(`Listening to port ${port}`);  
});
```

To ignite execution, run the terminal command npm start.

Creating Custom Middleware Elements

1. Request Logging Interceptor

A custom logging function can capture and print incoming traffic variables straight to the server console before routing takes place:

```
function logRequests(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // Pass control forward to the next middleware block in the execution sequence
}

// Register the logging interceptor globally across all route paths
app.use(logRequests);
```

2. Authentication Gatekeeper

This logic pattern evaluates inbound request header signatures to intercept unverified network traffic before it reaches restricted route paths:

```
function authenticateUser(req, res, next) {
  const isAuthenticated = req.headers['authorization'] === 'Bearer your_token_here';

  if (isAuthenticated) {
    next(); // Authorization verified successfully; proceed forward
  } else {
    res.status(401).send('Unauthorized'); // Halt request processing immediately
  }
}

// Option A: Apply globally across all routes
app.use(authenticateUser);

// Option B: Protect a single route by injecting it directly into the path definition
app.get('/protected', authenticateUser, (req, res) => {
  res.send('This is a protected route');
});
```

Custom Error-Handling Implementation

Error interceptor blocks require a four-parameter signature block arrangement (err, req, res, next) to ensure Express treats the script as an exception handler.

```
function errorHandler(err, req, res, next) {
  console.error(err.stack); // Log out full system trace points to assist debugging workflows
  res.status(500).send('Something went wrong!'); // Return a polished error notification to the client
}

// Critical rule: Register error handlers at the absolute base of app.js, following all routes
app.use(errorHandler);
```

⚠ Important: When an exception triggers inside a normal route block, developers must pass that failure block parameter forward explicitly by invoking `next(err)` to bypass standard route handlers and pass control straight to the custom error logic.



LEARNING ACTIVITIES / MGA GAWAIN SA PAGKATUTO

Activity 1: Setting Up an Express Application (Setup Foundations)

Create a new directory for your Express application. Initialize a new Node.js project using `npm init -y`. Install Express using `npm install express`. Create a file named `app.js` and set up a basic Express server that listens on a specified port. Define a simple route that responds with "Hello, World!" when accessed.

Activity 2: Implementing Built-in Middleware (Data Handling)

In the `app.js` file, add the `express.json()` and `express.urlencoded()` middleware to parse incoming request bodies. Create a POST route that accepts JSON data and responds with a confirmation message. Create another POST route that accepts URL-encoded form data and responds with the received data.

Activity 3: Creating Custom Middleware (Logic Injections)

Create a custom middleware function named `logRequests` that logs the HTTP method and URL of incoming requests. Create another middleware function named `authenticateUser` that checks for a specific authorization header. Apply both middleware functions to the Express application, ensuring that `logRequests` runs for all routes and `authenticateUser` runs for a specific protected route.

Activity 4: Implementing Error-Handling Middleware (Fault Tolerance)

Create a custom error-handling middleware function that logs the error stack and sends a user-friendly message. Introduce an intentional error in one of the routes (e.g., throw an error or call `next(new Error('Test Error'))`). Register the error-handling middleware at the end of the middleware stack in `app.js`.

Activity 5: Testing the Application (Suite Validation)

Use a tool like Postman or curl to send requests to the various routes you created. Test the logging middleware by observing the console output. Test the authentication middleware by sending requests with and without the required authorization header. Trigger the error in the application and observe how the error-handling middleware responds.

**GENERALIZATION**

Instructions: Synthesize your understanding by answering the following conceptual summary questions:

1. Explain how the runtime signature layout requirement (`err, req, res, next`) prevents Express from confusing an error interceptor with an application-level middleware function.
2. Identify the differences between built-in utility features like `express.static()` and external third-party middleware setups like `body-parser` or `morgan`.
3. Why does neglecting to call `next()` inside a custom authorization checker function cause client connections to hang indefinitely?

**ASSESSMENT**

Instructions: Complete the comprehensive evaluation requirements below.

Part I: Identification (10 Points)

1. The type of function that acts as an interceptor for requests and responses before they hit terminal path handlers.
2. The exact function name triggered to forward request control parameters to the next function block in sequence.
3. The global configuration method object code command utilized to register application-level middleware globally.

4. The exact number of parameter inputs required to form an operational error-handling custom script block.
5. The specific property name attached to requests where parsed incoming JSON data payloads become accessible.
6. The built-in Express middleware function executed to host raw static asset directories directly.
7. The third-party external tracking package explicitly chosen to print real-time HTTP interaction records.
8. The error object property command that outputs detailed code trace points to help developer debugging workflows.
9. The specific HTTP network response code string returned when authentication validation credentials fail.
10. The architectural design quality that allows long, complex programming rules to be broken down into small, distinct, reusable tasks.

Part II: Technical Code Debugging (10 Points)

An academic developer writes a custom logger tracking script but notices that client browser connections hang indefinitely whenever users request the homepage route. Analyze the source code block below, locate the structural mistake, and write the corrected code block requirement:

```
const express = require('express');
const app = express();

function systemTracker(req, res, next) {
  console.log('Traffic arrived: ${req.url}');
  // Structural Error point located here
}

app.use(systemTracker);

app.get('/', (req, res) => {
  res.send('Homepage Loaded Successfully.');
```



POSTTEST / PANGWAKAS NA PAGSUBOK

Instructions: Complete the comprehensive case diagnostic requirement block below.

System Chaining Design Challenge (20 Points)

A university administration portal tracks internal higher-education grades within a backend system layout. The supervisor wants to enforce a sequential chain of three independent middleware blocks globally before allowing users to review student profile scores:

1. Gatekeeper 1 (logTraffic): Logs the incoming timestamp, HTTP method, and route target to the server console.
2. Gatekeeper 2 (checkRole): Scans the inbound network request headers. If the x-user-role header is missing or does not match exactly matching criteria for 'Faculty', it stops the request and returns a 403 Forbidden error code.

3. Gatekeeper 3 (sanitizeInputs): Intercepts inbound payload data configurations via req.body and strips away characters that look like database command injections.

Required Evaluation Tasks:

1. Write the complete app.js code pattern showing how these three custom middleware functions are linked sequentially using the next() approach.
2. Create an intentional exception state trigger inside the protected grade path that routes systemic failures down to a centralized custom error-handling script block.
3. Detail how to construct a modular test request within a tool suite like Postman to verify that an unauthorized user triggers a proper security denial status response instead of bypassing your middleware chain.



REFERENCES / MGA SANGGUNIAN

- DigitalOcean. (2021). How to create custom middleware in Express.js. Retrieved from <https://www.digitalocean.com/community/tutorials/how-to-create-custom-middleware-in-express-js>
- Express.js. (n.d.). Using Express middleware. Retrieved from <https://expressjs.com/en/guide/using-middleware.html>
- Express.js. (n.d.). Writing middleware for use in Express apps. Retrieved from <https://expressjs.com/en/guide/writing-middleware.html>
- GeeksforGeeks. (2021). Middleware in Express JS. Retrieved from <https://www.geeksforgeeks.org/middleware-in-express-js/>
- Javatpoint. (n.d.). Express.js middleware. Retrieved from <https://www.javatpoint.com/expressjs-middleware>
- Simplilearn. (2022). Express JS Middleware: Everything you need to know. Retrieved from <https://www.simplilearn.com/tutorials/nodejs-tutorial/express-js-middleware>
- Verpex. (n.d.). Middleware in Express.js. Retrieved from <https://www.verpex.com/blog/web-development/what-is-middleware-in-express-js>